

Textul prezentării disertației (versiune optimizată ~9-10 min)

Slide 2 - Cuprins

În această prezentare voi vorbi:

În introducere, despre tema și scopurile lucrării

Apoi tehnologiile utilizate

Arhitectura generală a sistemului

Algoritmul Schnorr integrat în protocol

Fluxurile de înregistrare și autentificare

Proprietățile de securitate

Testarea funcțională și simularea atacurilor

Evaluarea performanței și comparațiile cu OAuth 2.0 și protocoalele PAKE

La final, concluziile și direcțiile de dezvoltare, urmate de bibliografie

Slide 3 - Introducere

Tema pornește de la o problemă structurală a aplicațiilor web: autentificarea clasică se bazează pe transmiterea parolei către server. Deși HTTPS reduce riscul interceptării, arhitectura rămâne vulnerabilă la compromiterea bazelor de date, reutilizarea parolelor și atacuri replay. OAuth 2.0, adoptat pe scară largă, nu rezolvă intrinsec această problemă.

Astfel, scopul este proiectarea unui protocol hibrid care îmbină schema Schnorr, bazată pe demonstrații cu cunoștințe zero, cu cadrul OAuth 2.0, astfel încât parola să nu părăsească niciodată dispozitivul clientului. Au fost implementate și trei fluxuri OAuth clasice ca referință, completate de o evaluare față de protocoalele PAKE consacrate, SRP-6a și zk-SNARK.

Slide 4 - Tehnologii

Python cu Flask a fost ales pentru server datorită ecosistemului de biblioteci criptografice. Flask-SQLAlchemy asigură persistența în SQLite cu trei tabele. Am utilizat cryptography și sympy pentru parametrii criptografici, PyJWT pentru jetoanele JWT semnate HS256, Authlib pentru OAuth 2.0 PKCE, iar bibliotecile Srp și pysnark pentru simularea SRP-6a, respectiv zk-SNARK.

Testarea se bazează pe pytest pentru cazurile funcționale și Locust pentru performanță sub încărcare concurentă, cu opt fire de execuție prin serverul waitress.

JavaScript a fost ales pentru client, unde toate calculele criptografice se execută local prin BigInt și Web Crypto.

Slide 5 - Arhitectură

Arhitectura este de tip trei niveluri. Nivelul de prezentare cuprinde interfața web unde se execută calculele ZKP pe dispozitivul utilizatorului, cu un monitor de rețea pentru auditare vizuală.

Nivelul de aplicație, conturat de Flask, gestionează logica criptografică, validarea apartenenței la subgrup, emiterea JWT și modulele OAuth 2.0. Securitatea este consolidată prin legarea de canal.

Nivelul de date conține baza SQLite. Nu se stochează niciun secret privat. Parametrii criptografici: P de 2048 de biți din RFC 3526, $G = 4$, securitate de aproximativ 112 biți.

Slide 6 - Algoritmul Schnorr

Schema Schnorr se bazează pe dificultatea logaritmului discret. Se utilizează un prim sigur $P = 2Q + 1$, generatorul G și cheia privată x din $\mathbb{Z}_Q$. Protocolul presupune: generarea cheii publice $y = G^x \bmod P$, transmiterea angajamentului $t = G^r \bmod P$, primirea provocării c , calculul răspunsului $s = (r + c * x) \bmod Q$ și verificarea egalității $G^s \bmod P = t * y^c \bmod P$.

Slide 7 - Fluxul de înregistrare

Clientul derivă local cheia privată din parolă prin scrypt cu un salt aleatoriu unic și calculează cheia publică y . Către server se transmite exclusiv cheia publică, validată prin verificarea apartenenței la subgrup și respingerea valorilor triviale.

Parola nu părăsește niciodată dispozitivul. La compromiterea bazei de date, derivarea cheii private echivalează cu rezolvarea logaritmului discret pe 2048 de biți.

Slide 8 - Fluxul de autentificare

Clientul generează un nonce aleatoriu r , calculează angajamentul t și îl transmite. Serverul validează apartenența la subgrup, creează o sesiune temporară de 5 secunde și returnează provocarea c .

Clientul recalculează cheia privată, calculează răspunsul s și îl transmite. Serverul verifică egalitatea matematică utilizând exclusiv cheia publică stocată, apoi emite un JWT ancorat de sesiunea ZKP. Sesiunea se invalidează imediat, iar mecanismul de excludere reciprocă asigură cel mult o sesiune activă per utilizator.

Slide 9 - Proprietăți de securitate

Rezistența la interceptare: ecuația $s = (r + c * x) \bmod Q$ nu poate fi rezolvată fără nonce-ul r , netransmis prin rețea. Protocolul oferă reziliență la atacuri Store Now, Decrypt Later.

Serverul stochează doar cheia publică, neutralizând breșele de date. Provocarea unică și invalidarea după consum previn replay-ul. Legarea de canal și expirarea JWT atenuează deturnarea sesiunii.

Auditul Wireshark a confirmat absența parolei în traficul de rețea.

Slide 10-11 - Validare și teste de securitate

Validarea acoperă 49 de cazuri pytest: 5 pozitive, 5 negative, 31 cazuri limită și 8 teste de securitate, cu rată de succes 100%.

Testele pozitive verifică absența parolei în baza de date și fluxurile complete. Testele negative acoperă autentificarea cu parolă greșită, replay, expirarea sesiunii și conflictele. Cazurile limită testează respingerea valorilor în afara Z_Q , valorile triviale, concurența pe 10 fire și tipuri de date neconforme. Testele de securitate validează simulatorul Schnorr, falsificarea fără cheie privată, izolarea sesiunilor și autentificarea cu cheia publică furată.

Simulările de atac includ: compromiterea bazei de date respinsă matematic, 10.000 de cereri fără coliziune, injectarea de parametri slabi prin MitM respinsă prin HTTP 422, și escaladarea forței brute de la 8 la 64 de biți demonstrând imposibilitatea computațională.

Slide 12 - Evaluarea performanței

Pe 100 de iterații: latență de verificare criptografică 33,87 ms, latență totală ZKP 64,76 ms, debit de 15,36 RPS secvențial și 17-19 RPS concurrent. Fluxurile OAuth 2.0 ating 440-505 RPS.

Compromisul este justificat de garanțiile criptografice superioare, iar latența rămâne imperceptibilă în autentificarea interactivă.

Slide 13 - Comparație ZKP față de OAuth 2.0

În ZKP Schnorr, secretul nu traversează rețeaua. În OAuth, parola este transmisă la autentificare. Latența ZKP: 64,76 ms, OAuth: sub 2,23 ms, raport aproximativ 1:67. Compromisul este justificat de eliminarea completă a riscului de exfiltrare a parolei.

Slide 14 - Comparație cu protocoalele PAKE

Pentru echitatea comparației, Schnorr a fost adaptat pe curba eliptică secp256r1 cu verificare mutuală și derivare ECDH. Rezultate pe 100 de iterații: pysnark 1,92 ms, SRP-6a 42,27 ms, Schnorr-EC 38,09 ms, Schnorr-EC-C-Lib 9,51 ms, Schnorr-Pow 155,91 ms.

Toate asigură netransmiterea parolei. SRP-6a oferă suplimentar rezistență la compromiterea bazei de date și standardizare prin RFC 5054. zk-SNARK are latența cea mai redusă, dar presupune o fază de configurare inițială ale cărei costuri nu sunt reflectate în măsurători.

Slide 15 - Direcții de dezvoltare

Ca direcții prioritare: derivarea secretului prin Argon2id, migrarea către curbe eliptice, reimplementarea nucleului criptografic în Rust sau C, explorarea algoritmilor post-cuantici, semnături asimetrice pentru jetoane, externalizarea sesiunilor către Redis, HTTPS obligatoriu și jetoane proof-of-possession.

Slide 16 - Concluzii

Implementarea demonstrează viabilitatea autentificării web fără transmiterea parolei. Protocolul satisface rigourile funcționale, de securitate și de performanță, validate prin 49 de teste cu succes 100%. Evaluarea comparativă poziționează soluția într-o zonă intermediară, iar varianta Schnorr-EC-C-Lib la 9,51 ms demonstrează potențialul de optimizare.

Ca limitări, parametrii actuali sunt adecvați exclusiv prototipului. Ca direcții viitoare se impun tranziția către curbe eliptice, Argon2id, limbaj compilat și rezistența la compromiterea bazei de date.

Slide 17 - Bibliografie selectivă

Acestea sunt câteva referințe din bibliografia utilizată.

Slide 18 - Mulțumesc

Vă mulțumesc pentru atenție!